

# CS 451 – Homework 2

## Reinforcement Learning: Mars Rover

Umut Ardil Murat  
Student ID: S028641

Spring 2026

### 1 Introduction

This report describes my implementations for Homework 2, where the goal is to teach a Mars Rover agent to navigate grid worlds using two different reinforcement learning approaches. In Questions 1 and 2, I implemented Value Iteration, which is a model-based planning algorithm that has access to the full transition model of the environment. In Questions 3 through 5, I implemented Q-Learning, a model-free algorithm that learns from experience without knowing the transition probabilities.

The project uses a Mars-themed grid world where the rover must reach sample collection sites (positive rewards) while avoiding craters (negative rewards). Movement is stochastic due to solar interference, meaning the rover has a chance of slipping sideways when it tries to move in a direction.

### 2 Q1: Value Iteration

Value Iteration is an offline planning algorithm. Since the agent has access to the full MDP (states, actions, transition probabilities, and rewards), it can compute the optimal value function before ever taking an action.

#### 2.1 Algorithm

The algorithm works by repeatedly applying the Bellman update across all states. At each iteration  $k$ , we compute:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

where  $T(s, a, s')$  is the transition probability,  $R(s, a, s')$  is the reward, and  $\gamma$  is the discount factor. After enough iterations, the values converge to  $V^*$  and we can extract the optimal policy by choosing the action that maximizes the Q-value at each state.

#### 2.2 Implementation Details

I implemented three methods in `value_iteration_agents.py`:

- `run_value_iteration()`: This method runs the main loop for `self.iterations` sweeps. The key point here is that updates must be done in batch. I create a new `Counter` object at the start of each iteration, fill it with the updated values using the old value function, and only replace `self.values` after the entire sweep is complete. Terminal states are skipped since their value is always 0.

- `compute_q_value_from_values(state, action)`: This computes  $Q(s, a)$  by iterating over all possible next states returned by the MDP's transition function. For each  $(s', p)$  pair, I accumulate  $p \cdot [R(s, a, s') + \gamma \cdot V(s')]$ .
- `compute_action_from_values(state)`: This returns the action with the highest Q-value. I iterate through all legal actions, compute their Q-values, and track the best one. If there are no legal actions (terminal state), it returns `None`.

One mistake I initially made was updating `self.values` in place during a sweep, which caused values computed later in the loop to use partially updated values from the same iteration. Switching to a separate `Counter` for each iteration fixed this.

### 3 Q2: Mission Profiles (Canyon Grid)

The Canyon Grid has a close exit worth +1, a distant sample depot worth +10, and a row of craters worth -10 along the bottom. The goal is to find parameter settings (discount  $\gamma$ , noise, living reward) that produce five different policies.

#### 3.1 Parameter Analysis

The three parameters each influence the policy in a specific way:

- **Discount ( $\gamma$ )**: Controls how much the agent values future rewards. A low discount makes the agent short-sighted, preferring the nearby +1 exit. A high discount makes the distant +10 depot worth pursuing.
- **Noise**: The probability of slipping sideways. When noise is 0, the cliff is completely safe since the rover always goes where it intends. With non-zero noise, walking next to the cliff becomes risky because the rover might slip into a crater.
- **Living reward**: The reward received at every non-terminal step. A negative value penalizes taking extra steps, pushing the agent toward the nearest exit. A positive value rewards being alive, which can make the agent prefer wandering indefinitely over reaching any terminal state.

#### 3.2 Chosen Parameters

Table 1 shows the parameter settings I chose for each behavior.

For part (e), the reasoning is that if the agent receives +1 at every step and the discount is close to 1, the sum of future living rewards approaches infinity, which is larger than any terminal reward. So the optimal policy is to avoid all exits and craters forever.

## 4 Q3: Q-Learning

Unlike Value Iteration, Q-Learning is a model-free algorithm. The agent does not have access to the transition function or the reward function. Instead, it learns by interacting with the environment and updating its estimates based on observed transitions.

### 4.1 Algorithm

The Q-Learning update rule for a transition  $(s, a, r, s')$  is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

|     | Behavior                   | $\gamma$ | Noise | Living Reward | Reasoning                                                                                                    |
|-----|----------------------------|----------|-------|---------------|--------------------------------------------------------------------------------------------------------------|
| (a) | Close exit, risk cliff     | 0.2      | 0.0   | 0.0           | Low discount makes +1 more attractive than the discounted +10. Zero noise means the cliff is safe.           |
| (b) | Close exit, avoid cliff    | 0.2      | 0.2   | 0.0           | Same low discount for the close exit, but noise makes the cliff dangerous so the agent takes the upper path. |
| (c) | Distant depot, risk cliff  | 0.9      | 0.0   | 0.0           | High discount makes the +10 worth the longer walk. No noise means the cliff path is safe.                    |
| (d) | Distant depot, avoid cliff | 0.9      | 0.2   | 0.0           | High discount for the distant depot, but noise makes the cliff risky so the agent goes around.               |
| (e) | Never terminate            | 0.9      | 0.2   | 1.0           | A living reward of +1 per step means infinite wandering beats any finite terminal reward.                    |

Table 1: Parameter settings for the five Canyon Grid policies.

where  $\alpha$  is the learning rate. This is a temporal difference update: the agent adjusts its Q-value estimate toward the observed sample  $r + \gamma \max_{a'} Q(s', a')$ .

## 4.2 Implementation Details

I implemented five methods in `qlearning_agents.py`:

- `get_q_value(state, action)`: Returns  $Q(s, a)$  from the Q-table. Since the table is a `Counter`, unseen state-action pairs default to 0.
- `compute_value_from_q_values(state)`: Returns  $V(s) = \max_a Q(s, a)$  by looking at all legal actions. Returns 0 for terminal states with no legal actions.
- `compute_action_from_q_values(state)`: Returns the action with the highest Q-value. When multiple actions tie for the best value, one is selected at random. This random tie-breaking is important because at the start of training all Q-values are 0, so without it the agent would always pick the same action and never explore the others.
- `update(state, action, next_state, reward)`: Applies the Q-learning TD update. I compute the old Q-value, the sample (reward plus discounted value of the next state), and update the Q-table in place.

An important detail is that unseen actions have a Q-value of 0 by default. This means that if all actions the agent has tried so far have negative Q-values, an untried action with Q-value 0 is technically the best. The implementation handles this correctly because `get_q_value` is used for all Q-value lookups, which goes through the `Counter` and returns 0 for unseen pairs.

## 5 Q4: Epsilon-Greedy Exploration

Pure greedy action selection (always picking the best known action) is problematic because the agent might never try actions that could lead to better outcomes. Epsilon-greedy addresses this with a simple strategy:

- With probability  $\epsilon$ : choose a random action from all legal actions.
- With probability  $1 - \epsilon$ : choose the best action according to current Q-values.

I implemented this in the `get_action` method using `util.flip_coin` to determine whether to explore and `random.choice` to pick the random action. When  $\epsilon = 0$ , the agent always exploits. As  $\epsilon$  increases, the agent explores more.

The same Q-Learning agent also works on the Crawler robot domain without any code changes, since the implementation does not make any assumptions about the format of states or actions. For the grid world, states are  $(x, y)$  tuples, while for the Crawler they are (arm angle, hand angle) tuples, but the Q-learning logic treats them all as opaque keys.

## 6 Q5: Q-Learning Deployment

For Q5, the Q-Learning agent is deployed on the `small_grid` with specific hyperparameters:  $\epsilon = 0.05$ ,  $\alpha = 0.2$ , and  $\gamma = 0.8$ . After training for 2000 episodes, the agent is tested for 100 episodes with  $\epsilon = 0$  (pure exploitation).

The agent achieves a win rate above 80% on the small grid, which confirms that the Q-learning implementation converges to a reasonable policy given enough training episodes. The same agent is also tested on the Crawler domain, where it successfully learns to move forward.

It is worth noting that tabular Q-Learning does not scale well to larger grids. Each state-action pair has its own independent Q-value entry, so the agent cannot generalize from one cell to another. On a larger grid, the agent would need many more episodes to visit and learn about every state, which is a fundamental limitation of tabular methods.

## 7 Conclusion

In this homework, I implemented two reinforcement learning algorithms. Value Iteration demonstrated how an agent with full knowledge of the environment can plan an optimal policy offline. The Canyon Grid analysis in Q2 showed how discount factor, noise, and living reward interact to shape the resulting policy.

Q-Learning showed how an agent can learn a good policy through trial and error, without knowing the environment model. The epsilon-greedy strategy provides a straightforward way to balance exploration and exploitation during training. The main limitation observed is that tabular Q-Learning does not generalize across states, which makes it impractical for larger state spaces.